DSL504: Natural Language Processing
Group Assignment 2 — Technical Report

Group 14

Kinshuk Gupta (12341190)
Harshit Kandpal (B24CS505)
Utkarsh Chattarjee (B24CS050)
Sujal Som (B24DS032)
Tanmay Jaiswal (B24DS033)

Indian Institute of Technology Bhilai

November 2026

Abstract

This report documents four experiments for DSL504 Group Assignment 2. **Q1 (NMT):** A Bahdanau-attention LSTM improves English–Hindi translation BLEU from 2.44 to 3.38 (+0.94) over a baseline encoder-decoder on 35,000 IITB training pairs. **Q2 (Sentiment):** A GloVe-BiGRU with additive attention achieves accuracy 0.5684 and macro-F1 0.5699 on five-class Amazon review rating prediction, outperforming a fine-tuned RoBERTa (accuracy 0.5380) trained for 3 epochs. **Q3 (Summarisation):** A 13.3M-parameter from-scratch Transformer trained for 110 minutes (6 partial epochs) on 287,113 CNN/DailyMail pairs achieves ROUGE-1/2/L/Lsum of 34.28/13.94/24.50/31.82 versus a Lead-3 baseline of 40.05/17.48/25.02/36.35; a source-ablation gap of 2.70 nats/token confirms genuine source usage. **Q4 (Generation):** A 2-layer LSTM (10.4M params) outperforms a Vanilla RNN (7.7M params) on human-rated story quality (3.81 vs. 3.10 overall) on TinyStories despite higher perplexity (21.17 vs. 18.30), demonstrating the likelihood trap; Top-k ($k = 40$, $T = 0.8$) is the recommended decoding strategy.

Contents

1 Introduction **2**

2 Q1: English–Hindi Neural Machine Translation **2**
2.1 Problem and Objective . . . 2
2.2 Data and Preprocessing . . . 2
2.3 Method . . . 2
2.4 Results . . . 2
2.5 Analysis . . . 3
2.6 Coverage and Limitations . . . 3

3 Q2: Amazon Review Rating Prediction **3**
3.1 Problem and Objective . . . 3
3.2 Data and Preprocessing . . . 3
3.3 Method . . . 3
3.4 Results . . . 4
3.5 Analysis . . . 4
3.6 Coverage and Limitations . . . 4

4 Q3: Abstractive Summarisation **4**
4.1 Problem and Objective . . . 4
4.2 Data and Preprocessing . . . 5
4.3 Method . . . 5
4.4 Results . . . 5
4.5 Analysis . . . 6
4.6 Coverage and Limitations . . . 6

5 Q4: The Plot-Twist Generator **6**
5.1 Problem and Objective . . . 6
5.2 Data and Preprocessing . . . 6
5.3 Method . . . 6
5.4 Results . . . 7
5.5 Analysis . . . 7
5.6 Coverage and Limitations . . . 7

6 Cross-Question Conclusions **8**

A Reproducibility Commands **9**

B Additional Figures **9**

1 Introduction

This report covers four NLP experiments exploring different aspects of sequence modelling: neural machine translation, sentiment classification, abstractive summarisation, and language-model-based story generation. All experiments use PyTorch with seed 42 (Q2 also uses seed 123). Experiments were conducted on CUDA GPU (Colab/Kaggle T4 or equivalent). The repository is organised as: `Question1/` (NMT), `Question_2/` (sentiment), `Question_3/` (summarisation), `Question_4/plot_twist_generator/` (story generation). A unified `requirements.txt` covers all dependencies.

2 Q1: English–Hindi Neural Machine Translation

2.1 Problem and Objective

We compare a baseline LSTM encoder-decoder with a Bahdanau additive-attention variant on the IITB English–Hindi parallel corpus [9].

2.2 Data and Preprocessing

Table 1: Q1 dataset statistics.

Property	Value
Training pairs	35,000
English vocabulary	7,224
Hindi vocabulary	8,214
Max English length	20 tokens
Max Hindi length	24 tokens

2.3 Method

Baseline: Stacked LSTM encoder-decoder; source encoded to a fixed-length context vector (final hidden state) used to initialise the decoder.

Attention model [1]: At each decoding step i an alignment score is computed for every encoder state h_j^{enc}:

$$e_{ij} = v^\top \tanh\!\left(W_1 h_j^{\text{enc}} + W_2 s_{i-1}^{\text{dec}}\right), \quad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}, \quad c_i = \sum_j \alpha_{ij} h_j^{\text{enc}}.$$

Hyperparameters: embedding 256, hidden 512, 2 layers, dropout 0.5, Adam, teacher-forcing 0.5, 8 epochs, batch 64, seed 42.

2.4 Results

Table 2: BLEU scores on the test set.

Model	BLEU
Baseline LSTM	2.44
Attention LSTM	**3.38**

Table 3: Length-bucket BLEU analysis.

Bucket	Count	Baseline	Attention	Δ
1–5 tokens	7	2.91	4.20	+1.29
6–10 tokens	122	2.27	3.38	+1.10
11–15 tokens	176	5.88	8.85	+2.97
16+ tokens	295	0.89	1.27	+0.39

Figures: `figures/model_comparison_curves.png`, `figures/sentence_length_analysis.png`.

2.5 Analysis

Bahdanau attention improves BLEU by 0.94 points (38.5% relative). Gains are largest in the 11–15-token bucket (+2.97 BLEU). Attention heatmaps (`figures/attention_heatmap_*.png`) show partially monotone alignments for short sentences; for long sentences attention diffuses to high-frequency Hindi function words (*ke*, *mein*), causing repetitive outputs. Five error cases (OOV collapse, word-order failure, missing word, lexical substitution, long-dependency diffusion) are documented in `scratch/results.json`.

2.6 Coverage and Limitations

Both models overfit after epoch 4 (validation loss rises while training loss continues to fall). BLEU scores are low due to small training corpus (35,000 pairs), high OOV rate, and Hindi morphological richness. Four qualitative heatmaps are saved; five error-analysis heatmaps are separate.

3 Q2: Amazon Review Rating Prediction

3.1 Problem and Objective

Five-class star-rating prediction (1–5) on Amazon product reviews. Two distinct systems: a GloVe-BiGRU with additive attention (recurrent baseline) and a fine-tuned `roberta-base` with masked-mean pooling [11].

3.2 Data and Preprocessing

200,000 training / 5,000 validation / 5,000 test reviews.

BiGRU pipeline: Lowercase, clean, concatenate title+body; training-only vocabulary (min-freq 3); cap at 200 tokens; GloVe-6B-100d initialisation; frozen epoch 1, unfrozen epoch 2.

RoBERTa pipeline: `RobertaTokenizerFast`, truncation at 256 tokens, dynamic padding; full backbone fine-tuning.

3.3 Method

BiGRU: 2-layer bidirectional GRU (hidden 256 per direction), additive attention pooling, dropout 0.3; Adam 1e-3, clip 1.0, early stop on validation macro-F1.

RoBERTa: Pretrained `roberta-base` fine-tuned end-to-end; attention-mask masked-mean pooling → dropout 0.2 → Linear(768, 5); AdamW (lr=2e-5, weight_decay=0.01), 8% linear warmup, fp16.

Seeds: 42 and 123.

3.4 Results

Table 4: Validation pooling ablation (RoBERTa, seed 42).

Pooling	Accuracy	Macro F1	MAE
Masked mean	0.5316	0.5246	0.6082
First token (`<s>`)	0.5302	0.5220	0.6102

Table 5: Held-out test results (mean ± std, 2 seeds).

Model	Accuracy	Macro F1	MAE
BiGRU + additive attention	0.5684 ± 0.0000	0.5699 ± 0.0008	0.5191 ± 0.0038
RoBERTa + masked mean	0.5380 ± 0.0037	0.5307 ± 0.0031	0.6123 ± 0.0021

Table 6: Per-class F1 (mean, 2 seeds).

Model	1	2	3	4	5
BiGRU	0.6423	0.4733	0.4751	0.5222	0.7368
RoBERTa	0.6421	0.4344	0.4007	0.4749	0.7014

Ordinal breakdown (seed 42): BiGRU exact/adjacent/distant = 56.84%/35.88%/7.28%; RoBERTa = 53.54%/35.52%/10.94%.

3.5 Analysis

Both models most confuse adjacent-star reviews. The BiGRU outperforms RoBERTa in this run; however, RoBERTa validation macro-F1 rises monotonically through epoch 3 for both seeds, indicating training had not converged. Five cases where RoBERTa is correct and the BiGRU is wrong are all 1-star reviews mislabelled as 2-star by the baseline. Two reverse cases (BiGRU correct) involve short, unambiguous negative reviews.

3.6 Coverage and Limitations

RoBERTa was not fully converged; additional training could improve results. BiGRU processes padded tokens (no packing), which may bias backward hidden states. Confusion matrix and ordinal breakdown use seed 42 only.

4 Q3: Abstractive Summarisation

4.1 Problem and Objective

Train a Transformer encoder-decoder from scratch on CNN/DailyMail 3.0.0 [6, 13]. Tokeniser: `bert-base-uncased` [3] (pretrained tokeniser only; no model weights). Evaluate with ROUGE [10] against Lead-3.

4.2 Data and Preprocessing

Table 7: Q3 dataset configuration.

Property	Value
Training pairs	287,113
Validation subset	3,000
Test set	11,490
Vocabulary size	30,522
Max source tokens	400
Max target tokens	128
Source truncation rate	87.2%
Seed	42

4.3 Method

Table 8: Transformer architecture.

Hyperparameter	Value
d_{model}	256
Encoder/Decoder layers	3 + 3
Attention heads	4
d_{ff}	1,024
Dropout	0.1
Tied embeddings	Yes
Total parameters	**13,343,232**
Optimizer	Adam (β=(0.9, 0.98), ε=1e-9) + Noam
Warmup steps	4,000
Batch size	32
Gradient clip	1.0
AMP (fp16)	Yes
Epochs	6 (epoch 6 partial: 50,009/53,838 steps)
Training time	110.1 minutes

Decoding: greedy (argmax), best checkpoint (epoch 6, lowest val loss) restored. Training was stopped by the 110-minute wall-clock budget; validation loss was still falling (undertrained, not converged).

4.4 Results

Table 9: ROUGE scores on the full 11,490 test set, greedy decoding.

System	R-1	R-2	R-L	R-Lsum
Our Transformer	**34.28**	**13.94**	**24.50**	**31.82**
Lead-3	40.05	17.48	25.02	36.35
Lead-3 (assignment ref.)	40.06	17.48	25.02	36.36

Source ablation: loss with true article 2.800, loss with wrong article 5.503, **gap = 2.70 nats/token** (confirms genuine source usage).

Training history: val loss 4.186 → 3.310 → 2.990 → 2.881 → 2.818 → 2.800 (monotonically decreasing; model undertrained).

4.5 Analysis

Lead-3 extracts the first three article sentences verbatim—a strong baseline on CNN/DailyMail because journalists front-load key facts. Our model generates abstractive text via greedy decoding but 68.1% of test outputs contain at least one repeated trigram, 9.3% contain hallucinated numbers (no copy/pointer mechanism), and 20.8% contain words not in the source article. The decoder cross-attention averaged over 4 heads shows 40.0% of attention mass in an 80-token window (source tokens 42–121); mean entropy 3.19 nats (vs. 5.99 for uniform over 400 tokens); 48% of attention peaks in the first 100 source tokens. The first summary sentence attends outside this window, confirming the model integrates information across the article head.

4.6 Coverage and Limitations

(1) Training was compute-limited: epoch 6 was partial (50,009/53,838 steps); LR had not fully decayed; val loss still falling. Additional training would likely improve ROUGE scores. (2) Greedy decoding without repetition penalty produces repeated outputs in 68.1% of cases. (3) No copy/pointer mechanism: hallucinated numbers appear in 9.3% of outputs. (4) 87.2% of source articles are truncated at 400 tokens. (5) Checkpoints excluded per assignment policy; regenerate via `summarizer.ipynb` (~110 min on T4).

5 Q4: The Plot-Twist Generator

5.1 Problem and Objective

Train a Vanilla Elman RNN (Model A) and an LSTM (Model B) [7] from scratch on TinyStories [4]. Compare perplexity, decoding-strategy diversity, and blinded human evaluation. Evaluate the "plot-twist challenge": generating continuations for adult-style prompts with models trained on children's stories.

5.2 Data and Preprocessing

Table 10: Q4 dataset statistics.

Property	Value
Training stories	20,000
Validation stories	1,000
Test stories	1,000
Vocabulary size	8,801 (min-freq 2)
Max sequence length	128 tokens (BPTT)
Val OOV rate	0.18%
Seed	42

Same tokeniser and vocabulary shared by both models; story-level splits.

5.3 Method

Both models share: embedding dim 256, hidden dim 512, 2 layers, dropout 0.2. Adam (lr=0.001), gradient clip 1.0, 15 epochs, patience 2.

Vanilla RNN [2]: $h_t = \tanh(W_{ih}e_t + W_{hh}h_{t-1} + b)$. Total params: 7,687,521.

LSTM [7]: Four-gate architecture with cell state highway mitigating vanishing gradients. Total params: 10,446,177.

Decoding strategies: greedy, temperature (0.5, 0.8, 1.2), Top-k (k=10, 40; T=0.8).

5.4 Results

Table 11: Training and perplexity results.

Model	Val PPL	Test PPL	Best Epoch	Train time (s)
Vanilla RNN	22.43	18.30	15	2,303
LSTM	25.16	21.17	15	4,788

Table 12: Decoding strategy diversity (10 prompts, Phase 3).

Model	Strategy	Distinct-2	Rep-4	Avg len
RNN	Greedy	0.236	0.730	100.0
RNN	Temp 0.8	0.902	0.007	98.4
RNN	Top-k (k=40, T=0.8)	0.875	0.009	94.0
LSTM	Greedy	0.594	0.284	58.4
LSTM	Temp 0.8	0.883	0.014	91.3
LSTM	Top-k (k=40, T=0.8)	0.909	0.008	78.3

Table 13: Blinded human evaluation (6 samples × 2 models, 3 raters, 1–5 scale).

Model	Coherence	Grammar	Relevance	Creativity	Overall
Vanilla RNN	2.78 ± 0.17	3.83 ± 0.15	2.89 ± 0.09	2.92 ± 0.23	3.10 ± 0.06
LSTM	3.79 ± 0.29	4.11 ± 0.17	3.53 ± 0.20	3.80 ± 0.19	$\mathbf{3.81 \pm 0.12}$

5.5 Analysis

The LSTM outperforms the Vanilla RNN on all human evaluation dimensions despite higher perplexity, empirically demonstrating the "likelihood trap". Greedy decoding is degenerate for both models (Rep-4 = 0.730 for RNN); Top-k ($k = 40$, $T = 0.8$) achieves the best diversity–fluency trade-off. Domain mismatch between children's training stories and adult plot-twist prompts causes both models to perform lexical concept laundering (OOV concepts projected to nearest children's-domain neighbours). A pretrained SmolLM2-135M extension provides a reference upper bound (outputs in `outputs/generations/llm_plot_twists.json`).

5.6 Coverage and Limitations

RNN/LSTM `.pt` checkpoint files are excluded from the repository (`.gitignore: *.pt`). Regenerate: `cd Question_4/plot_twist_generator && python src/run_phase2.py` (≈38 min RNN, ≈80 min LSTM on GPU; CPU subset via `train_stories_subset: 3000`, ≈15 min RNN). The vocabulary file `checkpoints/vocab.json` (176 KB) is included.

6 Cross-Question Conclusions

Attention across tasks. Soft attention (Bahdanau) in Q1 NMT enables inspectable source-to-target alignment; the same additive mechanism in Q2 sentiment highlights sentiment-laden tokens; Transformer cross-attention in Q3 shows concentrated but multi-peaked source coverage (entropy 3.19 nats vs. 5.99 uniform). Across tasks, attention mechanisms improve interpretability and often performance, but do not eliminate overfitting (Q1) or repetition (Q3).

Recurrent vs. Transformer. Q4 shows that an LSTM's gated cell state provides better long-range coherence than a Vanilla RNN, corroborating theoretical advantages of gated architectures. Q3 shows that even a small Transformer (13.3M params) can learn abstractive patterns from scratch, though it requires substantial compute to surpass a simple extractive baseline.

Metric-vs-human gaps. Q4 demonstrates the likelihood trap: lower perplexity (RNN: 18.30) does not imply better narrative quality (human overall 3.10 vs. LSTM's 3.81). Q3 shows ROUGE can be non-zero even for hallucinated summaries (45.1 for one hallucination case) while collapsing to 0 for repetition failures. Q2 shows that macro-F1 can favour models with more balanced per-class performance even if overall accuracy differs.

Limitations and future work. All experiments are compute-limited. Q3 would benefit from more training epochs, beam search with n-gram blocking, and a copy/pointer mechanism. Q4 would benefit from larger training data and larger model capacity. Q2 RoBERTa was not fully converged. Q1 requires a larger training corpus and subword tokenisation to reduce OOV rates.

References

References

[1] Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015. `arXiv:1409.0473`.

[2] Cho, K., et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. EMNLP 2014. `arXiv:1406.1078`.

[3] Devlin, J., et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL 2019.

[4] Eldan, R., & Li, Y. (2023). *TinyStories: How Small Can Language Models Be and Still Speak Coherent English?* `arXiv:2305.07759`.

[5] Fan, A., et al. (2018). *Hierarchical Neural Story Generation*. ACL 2018.

[6] Hermann, K.M., et al. (2015). *Teaching Machines to Read and Comprehend*. NeurIPS 2015.

[7] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.

[8] Holtzman, A., et al. (2020). *The Curious Case of Neural Text Degeneration*. ICLR 2020.

[9] Kunchukuttan, A., et al. (2018). *The IIT Bombay English-Hindi Parallel Corpus*. LREC 2018.

[10] Lin, C.-Y. (2004). *ROUGE: A Package for Automatic Evaluation of Summaries*. ACL Workshop 2004.

[11] Liu, Y., et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. `arXiv:1907.11692`.

[12] Luong, M.-T., et al. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. EMNLP 2015.

[13] Nallapati, R., et al. (2016). *Abstractive Text Summarization using Sequence-to-Sequence RNNs and Beyond*. CoNLL 2016.

[14] See, A., et al. (2017). *Get to the Point: Summarization with Pointer-Generator Networks*. ACL 2017.

[15] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 2017.

A Reproducibility Commands

```
# Setup
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Q1: Run question_1.ipynb on Colab/Kaggle with GPU

# Q2: Download GloVe-6B-100d, then run Question_2.ipynb

# Q3: Colab/Kaggle GPU, Run All (110 min):
#     export SUMMARIZER_PRESET=smoke  # for CPU test

# Q4: cd Question_4/plot_twist_generator
python src/run_phase1.py
python src/run_phase2.py   # ~38 min RNN, ~80 min LSTM on GPU
python src/run_phase3.py
python src/run_phase4.py
python src/run_phase5.py

# Extract facts (from repo root):
python report/extract_facts.py

# Generate this PDF:
python report/generate_pdf.py

# Build submission zip:
bash make_submission.sh
```

B Additional Figures

Additional figures are located in:

- `Question1/figures/` — NMT attention heatmaps and error analysis
- `Question_3/figures/` — Loss curves, LR schedule, ROUGE comparison, attention
- `Question_4/plot_twist_generator/outputs/plots/` — Loss curves, human evaluation, decoding strategies